وثيقة هندسية — تطبيق بالحكمة (Bilhikma Mobile)

شرح مُفصّل لملف تهيئة التطبيق AppBootstrap

`lib/core/app/app_bootstrap.dart`

الطبقة: Core / App | النوع: Static Utility Class | عدد الأسطر: 117 | نقطة الاستخدام: lib/main.dart

تاريخ الإعداد: 22-10-2026

1 نظرة عامة

`AppBootstrap` هو الطبقة المسؤولة عن **إقلاع التطبيق بشكل آمن**. فهو ليس واجهة ولا منطق أعمال، بل هو «غرفة تحكّم» تُشغَّل قبل `runApp()` وتضمن أمرين أساسيين:

1. **التقاط كل خطأ ممكن** في التطبيق — سواء وقع داخل إطار Flutter أو خارجه — وتسجيله عبر `AppLogger`.
2. **تنفيذ خطوات التهيئة بشكل معزول**، بحيث لا يؤدي فشل خطوة واحدة (التخزين المحلي، الترجمة، هوية الجهاز...) إلى توقّف التطبيق عن الإقلاع.

الفلسفة التي يقوم عليها الملف تُسمّى **الإقلاع المتدرّج (Degraded Start)**: من الأفضل أن يُقلع التطبيق ناقصَ ميزة على أن يبقى المستخدم أمام شاشة بيضاء. ولهذا فإن كل دوال التهيئة في هذا الملف *لا تُعيد رمي الاستثناء*، بل تُسجّله وتُسجّل اسم الخطوة الفاشلة ثم تُكمل.

> **الصنف مُغلق على ذاته**
>
> المُنشئ الخاص `AppBootstrap._()` يمنع إنشاء كائن من الصنف، وجميع الأعضاء `static`. هذا تعبير صريح عن أن الصنف خدمة عالمية (Namespace) لا حالة (State) له قابلة للتعدّد.

2 موقع الملف في المعمارية

الملف يقع في `lib/core/app/`، أي في الطبقة الأساسية المشتركة، ويُستدعى **حصراً** من `lib/main.dart`. وهو لا يعتمد على أي ميزة (feature) ولا على أي `Cubit` أو واجهة، بل يعتمد فقط على:

- `dart:async` — من أجل `TimeoutException` و`Future`.
- `dart:ui` — من أجل `PlatformDispatcher`.

- `flutter/foundation.dart` — من أجل `kDebugMode` و `kReleaseMode`.
- `flutter/material.dart` — من أجل `FlutterError` و `ErrorWidget` و `PaintingBinding`.
- `core/utils/app_logger.dart` — واجهة التسجيل الموحّدة.

هذا الاعتماد الأُحادي الاتجاه صحيح معمارياً: الطبقة الأساسية لا تعرف شيئاً عن طبقة العرض، والعلاقة لا تنعكس أبداً.

3 الثوابت وحالة الخطوات الفاشلة

```
// الاسم الذي تظهر به كل سجلات الإقلاع في وحدة التصحيح
static const String _logName = 'bootstrap';

// المهلة الافتراضية لكل خطوة تهيئة غير متزامنة
static const Duration stepTimeout = Duration(seconds: 12);

static final List<String> _failedSteps = <String>[];
static List<String> get failedSteps => List<String>.unmodifiable(_failedSteps);

static bool didStepFail(String name) => _failedSteps.contains(name);
```

العضو	الوظيفة والفائدة العملية
`_logName`	وسم ثابت لكل رسائل هذا الملف. يتيح تصفية سجلات الإقلاع وحدها في DevTools بكتابة `bootstrap`.
`stepTimeout`	سقف زمني افتراضي 12 ثانية لأي خطوة. يحمي من التعليق الأبدي إذا لم يرجع استدعاء أصلي (Native) أبداً.
`_failedSteps`	سجل داخلي بأسماء الخطوات التي فشلت فعلياً خلال هذه الجلسة.
`failedSteps`	قراءة فقط عبر `List.unmodifiable` — يمنع أي طبقة خارجية من تعديل السجل أو تلويثه.
`didStepFail(name)`	استعلام نقطي. يسمح لاحقاً لأي شاشة أو خدمة أن تسأل: «هل فشلت خطوة `localization`؟» وتتصرّف بناءً عليه (مثل إظهار شعار تنبيه أو تعطيل ميزة).

4 الدالة installErrorHandlers() — شبكة الأمان

هذه الدالة تُركّب **ثلاث طبقات التقاط** للأخطاء، ويضيف إليها `main.dart` طبقة رابعة. المجموع شبكة لا ينجو منها خطأ غير مُعالج.

الطبقة الأولى — FlutterError.onError

```
FlutterError.onError = (FlutterErrorDetails details) {
  AppLogger.error(
    'Flutter framework error in ${details.library ?? 'unknown library'}',
    name: _logName, error: details.exception, stackTrace: details.stack,
  );
  if (kDebugMode) FlutterError.presentError(details);
};
```

تستقبل أخطاء إطار Flutter نفسه: أخطاء `build` و`layout` و`paint` وتجاوزات القياس (Overflow) وما شابه. يُسجَّل اسم المكتبة التي أطلقت الخطأ مع المكدس الكامل. وملاحظة مهمة: `FlutterError.presentError` — أي الشاشة الحمراء الشهيرة — تُستدعى **في وضع التطوير فقط**، فلا يرى المستخدم النهائي شيئاً منها.

الطبقة الثانية — PlatformDispatcher.instance.onError

```
ui.PlatformDispatcher.instance.onError = (Object error, StackTrace stackTrace) {
  AppLogger.error('Unhandled error outside the framework',
      name: _logName, error: error, stackTrace: stackTrace);
  return true; // true = تم التعامل مع الخطأ، لا تُسقط التطبيق
};
```

تستقبل ما يقع **خارج** إطار Flutter: استثناءات `Future` غير المُلتقطة، وأخطاء القنوات الأصلية (Platform Channels)، وأخطاء المؤقّتات والمستمعين. إرجاع `true` هو القرار الحاسم هنا: إنه يُخبر المحرّك أن الخطأ استُهلك، فيستمر التطبيق في العمل بدل أن ينتهي.

الطبقة الثالثة — ErrorWidget.builder (الإصدار النهائي فقط)

```
if (kReleaseMode) {
  ErrorWidget.builder = (FlutterErrorDetails details) {
    AppLogger.error('Widget build failed', name: _logName,
        error: details.exception, stackTrace: details.stack);
    return const SizedBox.shrink();
  };
}
```

في الإصدار النهائي، إذا فشل بناء عنصر واجهة، يُستبدل العنصر المعطوب بمساحة فارغة غير مرئية (`SizedBox.shrink()`) بدل الشاشة الحمراء. النتيجة: يفقد المستخدم جزءاً صغيراً من الواجهة بدل أن تتحوّل الشاشة إلى رسالة خطأ مخيفة — والخطأ مُسجَّل كاملاً للمطوّر. وفي وضع التطوير يبقى السلوك الافتراضي ظاهراً حتى لا تُخفى المشاكل عن المبرمج.

> **نقطة تستحق الانتباه**
>
> الإخفاء الصامت للعنصر المعطوب ممتاز للعناصر الثانوية، لكنه قد يُنتج شاشة فارغة تماماً إذا كان الفاشل هو العنصر الجذري للصفحة. البديل الأنضج: إرجاع عنصر بديل صغير يحمل رسالة مترجمة عبر `LangKeys` بدل الفراغ المطلق.

الطبقة الرابعة — reportFatal عبر runZonedGuarded

```
// في main.dart
void main() {
  runZonedGuarded(_startApp, AppBootstrap.reportFatal);
}

// في app_bootstrap.dart
static void reportFatal(Object error, StackTrace stackTrace) {
  AppLogger.error('Fatal error escaped the startup zone',
      name: _logName, error: error, stackTrace: stackTrace);
}
```

هذه هي آخر شبكة. `runZonedGuarded` يُغلّف دالة الإقلاع بالكامل في «منطقة» خاصة، فأي خطأ يُفلت من الطبقات الثلاث السابقة — وخصوصاً أخطاء ما قبل تركيب المعالجات — يصل إلى `reportFatal` ويُسجَّل برسالة واضحة تُميّزه عن غيره.

الطبقة	نطاق الالتقاط	السلوك
`FlutterError.onError`	أخطاء إطار Flutter (بناء / تخطيط / رسم)	تسجيل + شاشة حمراء في التطوير فقط
`PlatformDispatcher.onError`	أخطاء غير متزامنة وأخطاء القنوات الأصلية	تسجيل + استمرار التطبيق
`ErrorWidget.builder`	فشل بناء عنصر واجهة (الإصدار النهائي)	تسجيل + عنصر فارغ بديل
`runZonedGuarded → reportFatal`	كل ما أفلت من الطبقات أعلاه	تسجيل كخطأ قاتل

5 الدالة runStep() — تنفيذ خطوة بمهلة وعزل

```
static Future<void> runStep(
  String name,
  Future<void> Function() step, {
  Duration timeout = stepTimeout,
}) async {
  try {
    await step().timeout(timeout);
  } on TimeoutException {
    _recordFailure(name, 'timed out after ${timeout.inSeconds}s',
        error: null, stackTrace: null);
  } catch (error, stackTrace) {
    _recordFailure(name, 'threw', error: error, stackTrace: stackTrace);
  }
}
```

هذه الدالة هي قلب الملف، وتقدّم ثلاث ضمانات في آن واحد:

- **العزل:** كل خطوة داخل `try/catch` خاص بها، فلا يُسقط فشلٌ واحدةٍ ما بعدها.
- **المهلة:** `.timeout(timeout)` تمنع التعليق اللانهائي، مع إمكانية تخصيص المهلة لكل خطوة.
- **الرصد:** كل فشل يُوثَّق باسمه وسببه — «تجاوز المهلة» أو «رمى استثناءً» — في `_failedSteps`.

التمييز بين `TimeoutException` وبقية الاستثناءات ليس تفصيلاً شكلياً: النوع الأول يعني «الخطوة ما زالت عاملة لكنها بطيئة»، والثاني يعني «الخطوة انكسرت» — وهما حالتان تُعالَجان بشكل مختلف تماماً عند التشخيص.

النسخة المتزامنة — runSyncStep()

```
static void runSyncStep(String name, void Function() step) {
  try { step(); }
  catch (error, stackTrace) {
    _recordFailure(name, 'threw', error: error, stackTrace: stackTrace);
  }
}
```

نفس المنطق للعمليات الفورية (تركيب مستمعين، ربط خدمة بأخرى). لا مهلة هنا — وهذا صحيح: الكود المتزامن لا يمكن مقاطعته بمهلة في Dart أساساً، فإضافتها ستكون وهماً لا حماية.

المُسجّل الموحّد — _recordFailure()

```
static void _recordFailure(String name, String what,
    {required Object? error, required StackTrace? stackTrace}) {
  _failedSteps.add(name);
  AppLogger.error('Startup step "$name" $what — starting without it',
      name: _logName, error: error, stackTrace: stackTrace);
}
```

نقطة واحدة لتسجيل كل الأعطال: تُحدّث القائمة وتكتب السجل بصيغة ثابتة. الرسالة «*starting without it*» تُعلن سياسة الملف بوضوح — التطبيق سيُكمل الإقلاع دون هذه الخطوة. استخدام `required Object?` ذكي: يسمح بتمرير `null` لحالة المهلة، لكنه يُلزم المُستدعي بالتصريح بذلك بدل السهو عنه.

6 الدالة tuneImageCache() — ضبط ذاكرة الصور

```
static void tuneImageCache({required bool isLowEndDevice}) {
  PaintingBinding.instance.imageCache
    ..maximumSizeBytes = (isLowEndDevice ? 24 : 56) << 20
    ..maximumSize      = isLowEndDevice ? 40 : 120;
  ...
}
```

القيم الافتراضية في Flutter لذاكرة الصور (نحو 100 ميغابايت و1000 صورة) سخيّة جداً وتُسبّب على الأجهزة الضعيفة قتلَ التطبيق من قبل النظام بسبب الذاكرة. هذه الدالة تضبط سقفين بحسب قدرة الجهاز:

المعيار	جهاز ضعيف	جهاز عادي
الحجم الأقصى بالبايت	24 ميغابايت	56 ميغابايت
أقصى عدد للصور المُخزّنة	40 صورة	120 صورة

العامل `<< 20` هو إزاحة ثنائية تساوي الضرب في 1048576، أي تحويل الميغابايت إلى بايت بتكلفة صفرية في وقت التنفيذ. ثم تُسجّل الدالة القيم المُطبَّقة فعلياً — مقروءة من الذاكرة نفسها لا من المتغيّرات — عبر `>> 20` للعودة إلى الميغابايت، وهو أسلوب تحقّق سليم يُثبت أن الضبط نُفِّذ بالفعل.

7 تسلسل الإقلاع الكامل في main.dart

1. **runZonedGuarded** — تغليف الإقلاع كله بشبكة الأمان الأخيرة
2. **WidgetsFlutterBinding.ensureInitialized** — تهيئة محرّك Flutter
3. **installErrorHandlers** — تركيب معالجات الأخطاء قبل أي عمل حقيقي
4. **Bloc.observer** — مراقب حالة الـ Cubit
5. **runStep('preferences')** — تهيئة التخزين المحلي (CacheUtil)
6. **runStep('localization')** — تهيئة EasyLocalization
7. **runStep('device_identity', 25s)** — هوية الجهاز بمهلة موسّعة
8. **runStep('service_locator')** — تسجيل التبعيات في GetIt
9. **tuneImageCache** — ضبط ذاكرة الصور حسب قدرة الجهاز
10. **runSyncStep × 3** — دورة الحياة، مراقبة الحساب، سياسة منع التصوير
11. **runStep('capture_policy')** — استعادة إعداد منع التصوير المحفوظ
12. **runApp(EasyLocalization → BilhikmaApp)** — إظهار الواجهة
13. **unawaited(runStep('push_notifications', 90s))** — الإشعارات بعد الإقلاع دون انتظار

لاحظ قرارين تصميميين مهمّين في هذا الترتيب. الأول: **تركيب معالجات الأخطاء في الخطوة الثالثة**، أي قبل أي تهيئة فعلية، حتى يكون أي خطأ لاحق مُغطّى. الثاني: **الإشعارات تُهيَّأ بعد** `runApp` **وبصيغة** `unawaited` ومهلة 90 ثانية، لأنها تتضمّن تواصلاً شبكياً مع الخدمة؛ فلا يجوز أن يبقى المستخدم أمام شاشة سوداء بسبب انتظارها. هذه هي الترجمة العملية لمبدأ الإقلاع المتدرّج.

8 خصائص التصميم الجيّدة في هذا الملف

الخصيصة	القيمة الهندسية
مسؤولية واحدة واضحة	الملف لا يفعل شيئاً سوى الإقلاع والأخطاء — لا منطق أعمال ولا واجهة.
عدم إعادة رمي الاستثناء	يضمن أن التطبيق يُقلع دائماً، ولو بميزات ناقصة.
مهلة لكل خطوة	يستحيل أن يتعلّق الإقلاع إلى الأبد.
تسجيل مركزي موحّد	كل الرسائل تحت الوسم `bootstrap` وبصيغة واحدة.
سلوك مختلف بحسب البناء	`kDebugMode` للمطوّر و`kReleaseMode` للمستخدم.
حالة للقراءة فقط	`failedSteps` غير قابلة للتعديل من الخارج.
وعي بقدرة العتاد	ضبط ذاكرة الصور بحسب `isLowEndDevice`.

9 ملاحظات وفرص تحسين

1. **عدم إرسال الأخطاء إلى خدمة مراقبة.** الأخطاء تُسجَّل محلياً فقط عبر `developer.log`، أي أنها تضيع في أجهزة المستخدمين. التحسين: إضافة قناة إبلاغ (Crashlytics / Sentry) داخل `AppLogger.error` في الإصدار النهائي — بتعديل مكان واحد، لأن التسجيل مركزي أصلاً.
2. **استدعاء `print` في `main.dart:55`.** السطر `print(CacheUtil.get(key: CacheKeys.deviceUuidKey))` يطبع معرّف الجهاز في السجلات. يُستحسن حذفه أو تحويله إلى `AppLogger.info` مقيَّد بـ `kDebugMode`، فهو حالياً تسريب بيانات تعريفية في الإصدار النهائي.
3. **`_failedSteps` لا تُفرَّغ أبداً.** لا توجد دالة لتصفيرها، وهذا مقبول في الإقلاع الواحد، لكنه يُربك أي محاولة لإعادة التهيئة أو في بيئة الاختبارات التي تشترك في الحالة الساكنة. إضافة `resetForTesting()` تحلّ المشكلة بسطر واحد.
4. **لا أحد يقرأ `failedSteps` حالياً.** الواجهتان `didStepFail` و`failedSteps` مكتوبتان ومُحكمتان لكنهما غير مُستخدمتَين في المشروع. استثمارهما مفيد: مثلاً تعطيل زر الإشعارات إذا فشلت خطوة `push_notifications`، أو إظهار تنبيه مترجَم إذا فشلت `preferences`.
5. **الصنف بلا اختبارات.** منطق `runStep` (نجاح / مهلة / استثناء) قابل للاختبار تماماً ولا يحتاج أي اعتماد خارجي، ويستحق اختبارات وحدة تُثبّت سلوكه.

10 خلاصة

`AppBootstrap` ملف صغير الحجم وكبير الأثر: 117 سطراً تُغطّي كل مسارات الخطأ الممكنة في التطبيق وتُحوّل الإقلاع من سلسلة هشّة قابلة للانقطاع إلى عملية مرنة تتحمّل الفشل الجزئي. الدوال الثلاث الأساسية (`installErrorHandlers` و`runStep` و`tuneImageCache`) تُعالج ثلاث مخاطر حقيقية في تطبيقات الهاتف: الانهيار غير المتوقّع، والتعليق عند الإقلاع، ونقص الذاكرة على الأجهزة الضعيفة.

التحسين الأهم المتبقي هو إيصال السجلات إلى خدمة مراقبة خارجية، لأن شبكة أمان لا يُقرأ ما تلتقطه تحمي المستخدم من الانهيار لكنها لا تُعلم الفريق بوجود المشكلة.
